Solution: Best Time to Buy and Sell Stock

Problem Statement

You are given an array `prices` where `prices[i]` is the price of a given stock on the ith day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Examples

1. **Input:** [3, 2, 6, 5, 0, 3]
   - **Expected Output:** 4
   - **Justification:** Buy the stock on day 2 (price = 2) and sell it on day 3 (price = 6). Profit = 6 - 2 = 4.
2. **Input:** [8, 6, 5, 2, 1]
   - **Expected Output:** 0
   - **Justification:** Prices are continuously dropping, so no profit can be made.
3. **Input:** [1, 2]
   - **Expected Output:** 1
   - **Justification:** Buy on day 1 (price = 1) and sell on day 2 (price = 2). Profit = 2 - 1 = 1.

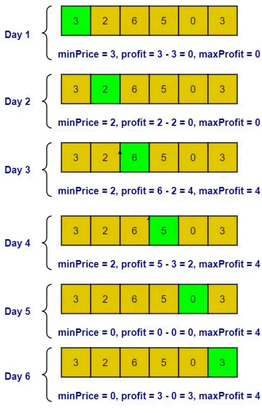
Solution

- **Understanding the Problem**:
  - The problem is to find the maximum difference between two numbers in an array, where the larger number comes after the smaller number.
- **Approach**:
  - Initialize two variables, `minPrice` and `maxProfit`.
  - Iterate through the list of prices.
    - For each price, calculate the profit that can be made by selling at that price (current price - `minPrice`).
    - Update `maxProfit` if the calculated profit is greater than the current `maxProfit`.
    - Update `minPrice` if the current price is less than `minPrice`.
- **Why This Works**:
  - This approach works by keeping track of the minimum price encountered so far and the maximum profit that can be made with that minimum price.
  - By continuously updating these two variables, the algorithm ensures that the maximum profit is always calculated relative to the lowest price encountered so far.

Algorithm Walkthrough

Consider the input `[3, 2, 6, 5, 0, 3]`:

- Initialize `minPrice` as `infinity` and `maxProfit` as `0`.
- Iterate through the list:
  - Day 1: price is `3`
    - `minPrice` is updated to `3`.
    - Profit = `3 - 3 = 0`. `maxProfit` remains `0`.
  - Day 2: price is `2`
    - `minPrice` is updated to `2`.
    - Profit = `2 - 2 = 0`. `maxProfit` remains `0`.
  - Day 3: price is `6`
    - `minPrice` remains `2`.
    - Profit = `6 - 2 = 4`. `maxProfit` is updated to `4`.
  - Day 4: price is `5`
    - `minPrice` remains `2`.
    - Profit = `5 - 2 = 3`. `maxProfit` remains `4`.
  - Day 5: price is `0`
    - `minPrice` is updated to `0`.
    - Profit = `0 - 0 = 0`. `maxProfit` remains `4`.
  - Day 6: price is `3`
    - `minPrice` remains `0`.
    - Profit = `3 - 0 = 3`. `maxProfit` remains `4`.
- The final `maxProfit` is `4`.



Code

```java
public class Solution {
    public int maxProfit(int[] prices) {
        // Initialize minPrice to the maximum possible integer value
        int minPrice = Integer.MAX_VALUE;
        // Initialize maxProfit to 0
        int maxProfit = 0;
        // Iterate through each price in the prices array
        for (int price : prices) {
            // Update minPrice to be the minimum of minPrice and the current price
            minPrice = Math.min(minPrice, price);
            // Update maxProfit to be the maximum of maxProfit and the difference between the current price and minPrice
            maxProfit = Math.max(maxProfit, price - minPrice);
        }
        // Return the final maxProfit
        return maxProfit;
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
        int[] example1 = {3, 2, 6, 5, 0, 3};
        int[] example2 = {8, 6, 5, 2, 1};
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

- **Time Complexity:** O(n), where n is the number of days. This is because the algorithm iterates through the list of prices once, performing constant-time operations for each price.
- **Space Complexity:** O(1), as it uses a constant amount of extra space (two variables to keep track of `minPrice` and `maxProfit`).